

exp-fips203-mlkem-pke-encrypt-k4

FIPS 203 Alg.14 ML-KEM-1024 PKE Encrypt 实现方案 (k=4)

ascendc 工程 (预研 / 交付示例)

2026-07-09

摘要

本文档描述 预研用例 `examples/incubating/exp-fips203-mlkem-pke-encrypt-k4/`: 在 Atlas A2 / Ascend910B4 上以 AscendC 实现 FIPS 203 Algorithm 14 (ML-KEM-1024, $K = 4$) **K-PKE.Encrypt** 全链。

生产 I/O (强制对齐 Alg.14):

- 输入: `ek_PKE` (1568 B) + 消息 m (32 B) + 随机性 `coins/r` (32 B); 另加与 `seed` 无关的静态 NTT/INTT LUT (非本算法中间态)。
- 输出: 仅密文 $c = c_1 || c_2$ (1568 B)。
- 禁止将 $\hat{A}$ 、 $y (\equiv r)$ 、 e_1 、 e_2 、 $\hat{t}$ 、 $\hat{y}$ 、 $\hat{u}$ 、 tr 、 u 、 v 、 μ 等中间量以 `.bin` 落盘到 `input//output/`, 或经 Host D2H/H2D 中转冒充全链。

自包含原则: AscendC 与 Host golden 源码全部 **vendored** 于本目录 (`prep/`、`compute/`、`scripts/`)。唯一允许的外部代码依赖: 仓库 `library/shared/`。禁止: 运行时 `#include/import` `ascendc-tests/`、其它 `examples/`、`frozen/`、`liboqs` (KAT 脚本除外)。

设备 Launch (与 PASS 探针 `pass-fix-f203-alg14-pke-encrypt-device-k4` 同构): SIM 2 次 (`prep` $\rightarrow$ `compute+` 内联 `pack`); CPU `tikicpu` 5 次 (`ASCENDC_SIM_HOST_MODE` 分叉)。中间 GM 仅设备内 **handoff**, 不落盘。SIM 910B4 参考 Total tick $\approx$ **626k** (`prep` $\approx$ 470k + `compute+tail` $\approx$ 155k)。

技术基线: `docs/notes/F203-Alg14-Encrypt-compute-tail-PASS`技术总结.md、`docs/notes/F203-Encrypt-compute-行18-19-UB`驻留技术总结.md、`docs/engineering/用例自包含与设备全链约束`.md。写法对齐 `exp-fips203-mlkem-pke-keygen-k4-实现方案-customspec.tex`。

目录

1 工程约束（自包含）

1.1 允许的外部依赖

路径	用途
library/shared/shake_x of_kernel/	AscendC SHAKE XOF 内嵌（prep SampleNTT / PRF）
library/shared/keccak_ f1600_kernel/	Keccak-f[1600]（若 Host/设备派生需要）
library/shared/	公共头（shake_general.h 等）
CANN / tikicpulib	工具链与仿真（非业务源码）

1.2 禁止的外部依赖与不安全 I/O

- **ascendc-tests** 任意探针目录的编译期 include / Python import（允许一次性 **vendor** 抄入本目录后切断外链）。
- **frozen/**、G5 correctness 拼装路线、liboqs 作为生产运行时依赖。
- 将 $\hat{A}$ 、 y/r 、 e_1 、 e_2 、 u 、 v 、 $\hat{t}$ 等写成 input/*.bin 或 output/*.bin 作为全链契约。
- Host 对拍以外的密码学计算冒充设备输出（如 Host 算 c 写回 output/c.bin）。
- prep $\leftrightarrow$ compute 之间 **D2H** 再 **H2D** 中转中间态。

1.3 唯一交付路径

本用例 **仅**下列生产路径；run.sh 默认即全量，**禁止**要求用户手动 export SIM_DIRECT=1 或一长串与 default 相同的宏：

组件	锁定配置
Prep $\hat{A}$ / CBD	与 prep 探针一致：F203_AHAT16_BLOCK_DIM=2，双 AIV 分片 SampleNTT；PRF+CBD $_{\eta=2}$ 产 $r\ e_1\ e_2$
Compute NTT/INTT	poly-batch；平面 mat_c；S1-S3 禁止 Gather ；行 18-20 UB 融合（见 MLKEM-NTT 定稿）
Tail pack	SIM 内联于 f203_encrypt_118_119；Compress $_{11/5}$ + ByteEncode $_{11/5}$
Host	main.cpp：SIM 2 launch / CPU 5 launch； ek+m+coins+LUT $\rightarrow$ 仅 c
资源友好	默认 ENCRYPT_SKIP_REBUILD=1、 CMAKE_BUILD_JOBS=2（对齐全链探针）

1.4 目录布局 (计划; 写码阶段落地)

```
exp-fips203-mlkem-pke-encrypt-k4/
  exp-fips203-mlkem-pke-encrypt-k4-实现方案-customspec.tex/.pdf # 本规格
  main.cpp # 生产 Host: 2/5 launch
  f203_encrypt_full_layout.h # prep compute GM handoff
  f203_encrypt_prep_entry.cpp # Launch 1
  prep/ # vendored 行 3-15
  compute/ # vendored 行 2/16-22 (含内联 pack)
  cmake/ 或 CMakeLists.txt
  scripts/gen_data.py # 自包含: ek/m/coins/lut + golden/c
  scripts/host_golden/ # golden_encrypt / gen_ek_pke
  run.sh # 默认 SIM_DIRECT=1 + SKIP_REBUILD
  SELF_CONTAINED.md # 写码时补
  STATUS.md # 写码验收后补
```

参考实现过程 (只读, 写码时 vendor, 禁止运行时外链): ascendc-tests/pass-fix-f203-
alg14-pke-encrypt-device-k4/。

2 数学语义 (Alg.14, $K = 4$, $N = 256$, $q = 3329$)

2.1 参数与符号

- 参数集: **ML-KEM-1024** (FIPS 203; $k = 4$; $d_u = 11$; $d_v = 5$; $\eta = 2$)。
- c_1 长度: $k \cdot 352 = 1408$ B; c_2 长度: 160 B; $|c| = 1568$ B。
- ρ : 取自 **ek_PKE** 尾 32 B (偏移 [1536 : 1568))。

2.2 输入输出 (生产契约)

路径	尺寸	说明
input/ek_pke.bin	1568 B	公钥; 含 ρ
input/m.bin	32 B	消息
input/coins.bin	32 B	加密随机性 (Alg.14 的 r 种子)
input/lut_*.bin	4×64 KB	静态 NTT/INTT even/odd planar-stacked; 与 seed 无关
output/c.bin	1568 B	唯一产物 $c = c_1 c_2$

可选 (非产物, 仅 CPU 分段路径注入): input/golden_v.bin (1024 B) ——tikicpu 无 $k=8$ INTT 时注入 v 供 pack; SIM 全设备**禁止**依赖。调试 dump (须显式 env, 非默认) 可 D2H 中间量, **不得**写入标准验收契约。

2.3 流水线 (FIPS 行 1–22)

1. 行 1: $N \leftarrow 0$ (无运算; PRF nonce 从 0 起)。
2. 行 2: $\hat{t} \leftarrow \text{ByteDecode}_{12}(\text{ek_PKE})$ (设备 compute 核内; Host 不预解码落盘)。
3. 行 3–7: $\rho \leftarrow \text{ek_PKE}[1536 : 1568]$; $\forall(p, j): \hat{A}[p, j] \leftarrow \text{SampleNTT}(\rho \| j \| p)$ 。
4. 行 8–15: $\text{PRF}(\text{coins}, n) + \text{SamplePolyCBD}_{\eta=2} \rightarrow r \| e_1 \| e_2$ (扁平 `re[9, 256]`)。
5. 行 16–18: $\hat{y} \leftarrow \text{NTT}(y)$, $y \equiv r$; 内积 $(\hat{u}, \text{tr}) \leftarrow (\hat{A}^\top \mid \hat{t}^\top) \circ \hat{y}$ 。
6. 行 19–21: $u \leftarrow \text{INTT}(\hat{u}) + e_1$; $e'_2 \leftarrow e_2 + \mu(m) \bmod q$; $v \leftarrow \text{INTT}(\text{tr}) + e'_2$ 。
7. 行 22: $c_1 \leftarrow \text{ByteEncode}_{11}(\text{Compress}_{11}(u))$; $c_2 \leftarrow \text{ByteEncode}_5(\text{Compress}_5(v))$; $c \leftarrow c_1 \| c_2$ 。

本阶段不做: KEM Encaps (Alg.20)、Decaps、KeyGen; prep+compute 单 kernel 融合 (远期); 去掉静态 LUT 文件的 ROM 路线 (已关闭, 见 qa 2026-07-09)。

2.4 Golden (I/O 等价)

- 锁死种子 `SEED_D=20260619` (与全链 PASS 探针一致)。
- Host: `scripts/host_golden/golden_c.py` (自包含) $\rightarrow$ `golden/c.bin`。
- 验收: `output/c.bin` vs `golden/c.bin` `max_abs_diff=0` (CPU $\wedge$ SIM)。
- 禁止以「与探针源码同构」为验收; 仅 I/O 等价。

3 编排与 Launch

3.1 SIM (生产路径, ASCENDC_SIM_HOST_MODE)

Session 1 aclrtStream

```
Launch 1  f203_encrypt_prep      blockDim=2  AIV_ONLY
          in:  ek_pke, coins
          out: a_hat, re          # 仅 GM, 不落盘
```

```
Launch 2  f203_encrypt_l18_l19   blockDim=1  MIX_AIC_1_2
          in:  ek_pke, m, a_hat, y/e1/e2(=re 切片), ws+LUT
          out: c (内联 tail pack); u/v 可写 GM 但不 D2H
```

合计 2 launch; 参考 tick $\approx$ 626k。

3.2 CPU (tikicpu)

#	kernel	说明
1	f203_encrypt_prep	产 a_{hat}, re (GM)
2	f203_encrypt_ntt_y	MIX 段: $NTT(y)$
3	f203_encrypt_at_jp	内积
4	f203_encrypt_intt_e1	$INTT + e_1 \rightarrow u$
5	f203_encrypt_alg14_pack	$v = \text{golden}_v$ 注入; $pack \rightarrow c$

CPU 路径**不是**与 SIM 同构的完整设备 Encrypt (无融合 $k=8$ INTT); 验收仍以 $c_{\text{max}}=0$ 为准。写码须用 ASCENDC_BUILD_CPU/ASCENDC_BUILD_SIM + ASCENDC_SIM_HOST_MODE, 禁止 per-probe 宏分叉。

3.3 GM handoff (单 arena)

INPUTS (H2D 一次): $ek_{\text{pke}}[1568]$ $m[32]$ $coins[32]$ + LUT $\rightarrow$ ws
 Launch1 WRITES / Launch2 READS: $a_{\text{hat}}[16, 256]$ $i32$; $re[9, 256]$ $i32$
 $re: \text{poly}0..3=r_y; 4..7=e_1; 8=e_2$
 Launch2: 设备内 u, v ; 唯一 D2H = $c[1568]$
 SCRATCH: ws_{compute} 331776 B (含 LUT 段)

禁止 Host 将 re D2H 再拆成三个 buffer H2D。

4 AI Core 规模

段	核类型 / blockDim / 角色
Prep	KERNEL_TYPE_AIV_ONLY; blockDim=2; 双 AIV 分片 SampleNTT (各 8 poly); PRF+CBD 编排同 prep 探针
Compute+pack	KERNEL_TYPE_MIX_AIC_1_2; blockDim=1; 1 AIC + 2 AIV ; NTT/INTT Stage2 在 AIC; AIV 做 split/pack/内积向量段与内联 pack

内部命名: $\text{prep aiv}=2$; $\text{compute aicore}=1$ (MIX 1:2)。**选型理由**: 与已 PASS 全链探针同构, 避免重开 FSM/分核实验; 数据并行维为 16 SampleNTT poly (prep) 与 poly-batch NTT (compute)。**禁止**实现擅自改 blockDim 却不回写本规格。

串行占用: $\text{prep } 2$ Vector AI Core; $\text{compute } 1$ 颗 AI Core (Cube+2 Vector)。CPU [SUCCESS] [AIC_*] 为 tikicpu artifact; 以 SIM $\text{profile}_*.toml$ / Total tick 为准。

5 数据与组件同步规划

- prep 核内: UB/GM + SHAKE 内嵌同步 (对齐 KeyGen prep 笔记)。

- prep→compute: 仅 Host 顺序 launch + 同 session GM 指针; 无文件交接。
- compute 内: MIX CrossCore / FSM (NTT→内积→INTT); $e_2+\mu$ 前缀折叠; pack 在 AIV 分支按 subBlock 分片写 c 。
- 原理: docs/notes/F203-Encrypt-compute-行18-19-UB驻留技术总结.md (INTT 输入须来自内积当次 UB)。

5.1 实现映射 (写码时)

main.cpp

```
-> Launch1: f203_encrypt_prep_entry / prep/*
-> Launch2: compute/f203_encrypt_l18_l19_kernel.cpp (+ pack ops)
-> D2H: c only
```

f203_encrypt_full_layout.h

```
-> static_assert prep a_hat/re 尺寸 == compute 期望
```

6 全量 API 清单 (查阅 CANN 9.0.0)

说明: 分类按 PDF 第 2 章; 在线页为社区 atlasascendc_api_07_*. 本表列本用例拟用主路径 API; 细粒度算子实现已在上游 PASS 探针验证, 写码以 vendor 后本目录源码为准, 并遵守 Rule 「写 AscendC 前查索引」。

API	分类	在线文档 ID	A2	输入/输出 dtype	备注
GlobalTensor	2.2	07_0007	✓	GM 描述	SetGlobalBuffer
LocalTensor	2.2	07_0006	✓	UB 描述	Alloc/Get
GetBlockIdx/G etSubBlockIdx	2.3 系统	07_0185 邻 域	✓	核/子核索引	prep 分片; MIX 角色
KERNEL_TASK_T YPE_DEFAULT	Utils	编程指南	✓	AIV_ONLY / MIX_AIC_1_2	两核类型
TPipe/TQue/TB uf	2.3 资源	07_0108/0137	✓	UB 管道	向量段
DataCopy	2.3.1	07_0101	✓	int8/int16/int32 GM↔UB	对齐约束
DataCopyPad	2.3.1	查阅索引	✓	非整块	按需
PipeBarrier	2.3 同步	查阅索引	✓	—	UB 可见性
CrossCoreSetF lag/CrossCore WaitFlag	跨核	API 列表	✓	MIX AIC↔AIV	NTT/INTT FSM
Add/Sub/Muls/M ul	2.3.3	07_0035– 0055	✓	int16/int32	CBD/NTT/内积 辅助

API	分类	在线文档 ID	A2	输入/输出 dtype	备注
ShiftRight/Sh iftLeft	2.3.3	07_0059	✓	int32	limb/编码
Cast	2.3.3	07_0073	✓	见 Cast 链	A2 无直连 int32→int8
Mins	2.3.3	07_0057	✓	int16	Alg.7 rej
GatherMask	2.3.3.4	07_0071	✓	掩码收集	rej compact (非 NTT S1-S3)
Gather	2.3.3	07_0071 邻 域	✓	—	仅 允许 NTT 后 basemul/编码等; 禁止 Tag5T S1-S3 平面 mat_c Gather
Compare/Compa res	2.3.3.4	07_0066/0068	✓	见索引	int32 仅 EQ; bit 打包勿当逐 lane 布尔
GetCmpMask	2.3.3.4	07_0223	✓	uint8 掩码	与 Compare 寄存 器版配合
Cube MMAD / AicMmad 路径	2.3.2 矩 阵	教程 + 工程 封装	✓	int8×int8→int32	Stage2; B 矩阵为 ws 内 LUT

6.1 评估过但未采用 / 已关闭

项	原因
AIC 直读 __gm__ const LUT 作 MMAD B	SIM 挂死 (路线 11, qa 2026-07-09); 仍用 ws+LUT H2D
ByteEncode _{5/11} 真 · 向量 pack (Gather 拼字)	比标量更慢; 采用 Compress 向量 + Encode 标量 pack
NTT S1-S3 内 Gather correctness G5 多段拼装 中间态 bin staging ($\hat{A}/u/v$)	MLKEM Tag5T 禁令; 平面 mat_c + UB 融合 已由 prep+compute-tail 两上游取代; 禁止抄 G5 违反 Alg.14 I/O 与安全落盘约束

7 数学步骤 → API 覆盖率

数学步骤	主路径	覆盖	缺口/备注
SampleNTT / rej	SHAKE 内嵌 + Mins/GatherMask	高	对齐 KeyGen/prep 探针

PRF + CBD $_{\eta=2}$	向量 MTE/V (prep)	高	同 prep 探针
ByteDecode $_{12}(\text{ek})$	设备 AIV0	高	核内; 不落盘 $\hat{t}$
NTT(y) S1-S3	向量 split + AIC	高	无 Gather;
	MMAD + pack		poly-batch
内积 $kP=5$	向量/UB	高	UB 驻留喂 INTT
INTT + e_1/e'_2	MIX + Add	高	$e_2+\mu$ 前缀折叠
Compress $_{11/5}$	向量 Barrett/Cast	高	见 compress 探针选型
ByteEncode $_{11/5}$	标量 pack	部分	有意标量; 非缺口隐瞒
$c_1\ c_2$ 写出	DataCopy GM	100%	唯一 D2H

主路径: 非宣称全链 100% 向量 (Encode $_{5/11}$ 标量 pack 已定型); NTT/内积/Compress 等关键段与 PASS 探针一致。

8 验证

8.1 生产验收 (默认; 写码完成后)

```
cd examples/incubating/exp-fips203-mlkem-pke-encrypt-k4
bash run.sh -r cpu -v Ascend910B4
bash run.sh -r sim -v Ascend910B4
# 期望: 两模式 [cmp] c max=0; output/ 仅 c.bin; SIM 根目录 0 stray dump
# 参考 tick 626k
```

禁止标准文档要求手动 SIM_DIRECT=1 或与 default 重复的 HAT_* export。

8.2 自包含 golden

scripts/gen_data.py: 优先可复用已知正确 fixtures; 缺失时本目录用 SEED_D=20260619 自生成 ek/m/coins/golden/c + 静态 LUT。

8.3 可选 liboqs KAT

交付前可选脚本对照 (非 run.sh 默认); 不得把 liboqs 链进生产二进制。

9 性能 (SIM 910B4 参考)

上游全链探针 (2026-07-08/09): Total tick $\approx$ **626139–626227** (2 launch)。本 exp 写码验收目标: 与上述同量级; **远超** (例如数分钟无日志) 视为回归, 不得靠拉长防挂死预算掩盖。

10 写码衔接（本规格闭环后）

1. 用户确认本 customspec 路径后，执行 **【预研】**：按本规格 vendor 探针实现到本目录并自包含化。
2. 活跃 customspec: `examples/incubating/exp-fips203-mlkem-pke-encrypt-k4/exp-fips203-mlkem-pke-encrypt-k4-实现方案-customspec.tex`（及同名 PDF）。
3. 晋级 `examples/stable/stable-fips203-mlkem-pke-encrypt-k4` 须另走 # 交付 #，且须 baseline-registry 齐备。